

CSC3210 - Quiz - File Descriptors, Pipes, and Message Queues

Name: Solution

Answer the questions in the spaces provided. Work is to be done on your own without a computer/phone. Justify your answers and show all work where appropriate.

1. (10 points) After a `fork()`, the parent and child processes have separate address spaces but share entries in the file descriptor table. Explain what “sharing entries” means in practice, including what happens to the file position pointer. Support your answer with an example (code is not required).

Solution: After `fork()`, the child inherits copies of the parent’s open file descriptor numbers, but both the parent’s and child’s file descriptor table entries point to the same underlying kernel object. This means they share the same file position pointer. They do not share user-space memory, but they do share the kernel-managed file state.

For example, if a parent process writes data to the file using the file descriptor, then the data will be written and the file position pointer will be advanced to the location in the file at the end of the data. The child process will see the file position pointer in its new location.

2. (10 points) What is a pipe, and how does it differ from a regular file in terms of where data is stored and how the file position pointer is managed?

Solution: A pipe is a unidirectional data channel managed entirely by the operating system. Data is stored and managed in operating system memory, not on disk. Unlike a file, the programmer does not manage the file position pointer. Data written to the write end comes out the read end in order, and once read, it is consumed and gone. The OS handles all internal positioning.

CSC3210 - Quiz - File Descriptors, Pipes, and Message Queues

3. (10 points) When using a pipe with multiple processes created with fork, why is it a good practice for processes to close the ends of the pipe when they are no longer needed? Support your answer with an example (code is not required).

Solution: Closing the ends of the pipe tells the operating system that the process no longer needs to use the pipe (for either reading or writing depending on which end is closed).

For example, when all write ends of the pipe are closed, the operating system knows that it is not possible for any more data to be written to the pipe. At that point, the operating system can send end of file to any process reading on the pipe.

If all processes close all ends of the pipe (both reading and writing), then the operating system knows it can release any storage allocated for the pipe.

4. (10 points) List two (2) ways that POSIX message queues differ from pipes.

Solution:

1. Message queues are structured: messages must be sent and received as complete, fixed-size units (atomic operations), while pipes are unstructured byte streams.
2. Message queues are named and persistent: they survive process termination and are identified by a string name, while anonymous pipes are temporary.
3. Message queues support message priorities: higher-priority messages are delivered first, while pipes deliver data strictly in FIFO byte order.